

ECOM EXPERTS

Ecom editcartadmin

Manual de usuario, tecnico y de desarrollo

ecom_editcartadmin 1.0.0 · 07/09/2026

Contenido

¿Qué hace?

Instalación

Cómo abrir el editor

Qué puedes hacer

Avisos legales importantes

Preguntas frecuentes

1. Propósito

2. Arquitectura

3. API del core utilizada (verificada en tags 1.7.8.11 / 8.1.7 / 9.0.x)

4. Endpoints AJAX

5. Seguridad

6. Configuración (Configuration)

7. Estructura de ficheros

Flujo general

Añadir un nuevo endpoint AJAX

Reglas y convenciones (proyecto)

Integración con la página nativa de carritos

Pruebas recomendadas

Puntos a verificar en la instalación real (marcados en la investigación)

¿Qué hace?

Permite editar el carrito de compra de un cliente desde la administración: añadir y quitar productos, cambiar cantidades, aplicar vales y descuentos, activar envío gratis y marcar el carrito como regalo. Todo se recalcula al instante.

Instalación

1. Sube la carpeta `ecom_editcartadmin` a `/modules/` de tu tienda.
2. Ve a **Módulos** → **Gestor de módulos**, busca "Editar carritos (Admin)" e instálalo.
3. (Opcional) Entra en su configuración para activar/desactivar el botón en la lista de carritos y otras opciones.

Cómo abrir el editor

Tienes dos formas:

- **Menú:** *Pedidos* → *Editar carritos*. Escribe el **ID del carrito** y pulsa **Cargar carrito**.
- **Botón en la lista de carritos:** en *Pedidos* → *Carritos de compra*, cada carrito muestra un botón **Editar carrito** que abre el editor directamente.

Qué puedes hacer

- **Cambiar cantidades:** edita el número en la columna *Cantidad*. Si pones 0, se elimina la línea.
- **Quitar un producto:** botón de papelera en cada fila.
- **Añadir un producto:** en el bloque *Añadir producto*, escribe nombre o referencia. Aparece una lista con **imagen, precio y stock**, y cada resultado tiene un botón **Añadir**. Si el producto tiene combinaciones (talla, color...), se muestran como opciones separadas. Indica la cantidad antes de añadir.
- **Vales:** introduce el código del vale y pulsa *Aplicar*.
- **Descuento rápido:** elige *Porcentaje %* o *Importe fijo*, indica el valor y pulsa *Aplicar*.
- **Envío gratis:** botón *Aplicar envío gratis*.
- **Quitar un descuento:** botón rojo junto a cada descuento aplicado.
- **Marcar como regalo:** casilla en *Otras opciones*.
- **Totales:** el panel de la derecha muestra productos, descuentos, envío y total, actualizados en cada cambio.

Avisos legales importantes

- Edita el carrito **solo a petición o con el consentimiento del cliente**. Estás tratando datos personales (RGPD) y estás sujeto al deber de confidencialidad.

- El cliente debe poder **ver y confirmar el precio final antes de pagar**; no le impongas cambios sin su conocimiento.
- Un carrito **ya convertido en pedido** es un contrato formalizado: el módulo lo bloquea por defecto, porque modificarlo puede requerir una **factura rectificativa**.
- Cada cambio se **registra** (empleado, fecha y acción) y queda disponible en *Parámetros avanzados* → *Logs*.

Estos avisos son informativos. El responsable del cumplimiento legal y del tratamiento de datos es el titular de la tienda.

Preguntas frecuentes

- **No aparece el botón en la lista de carritos:** comprueba que la opción está activada en la configuración del módulo. En PrestaShop 9, el botón se añade mediante JavaScript; si tu tema de administración personaliza mucho la tabla, usa el menú *Pedidos* → *Editar carritos*.
- **No me deja editar un carrito:** probablemente ya está convertido en pedido. Puedes permitirlo en la configuración (no recomendado).
- **El buscador no encuentra el producto:** escribe al menos 2 caracteres; busca por nombre o por referencia.

Edit Cart Admin — Documentación Técnica

Módulo: `ecom_editcartadmin` · **Clase:** `Ecom_Editcartadmin` · **Versión:** 1.0.0 **Compatibilidad:** PrestaShop 1.7.x, 8.x y 9.x · **PHP:** 7.1 – 8.x **Autor:** Ecom Experts · **Licencia:** AFL-3.0

1. Propósito

Permite editar el contenido de un carrito de compra desde el back-office: añadir/quitar productos, cambiar cantidades, buscar productos y combinaciones (con imagen), aplicar vales por código, crear descuentos rápidos (porcentaje o importe fijo), activar envío gratis y marcar el carrito como regalo. Incluye avisos legales (RGPD/consumo) y registro de auditoría de cada cambio.

2. Arquitectura

- **Núcleo:** controlador propio legacy `AdminEcomEditCartController` (`ModuleAdminController`), 100 % compatible 1.7/8/9. Renderiza una plantilla Smarty propia y expone endpoints AJAX `ajaxProcessXxx`. El render dinámico de la interfaz lo hace `views/js/editcart.js` a partir del estado JSON que devuelve cada endpoint (una sola fuente de verdad).
- **Integración con la página nativa de carritos:** no existe hook de display en la vista `sell/orders/carts/{id}/view` (verificado en el core de PS 8/9). Por eso el botón "Editar carrito" se inyecta por JavaScript vía `hookDisplayBackOfficeHeader` (único punto común fiable en 1.7/8/9), que detecta la página de carritos y añade el botón enlazando al controlador con el `id_cart`.
- **Override opcional:** `override_v8/controllers/admin/AdminCartsController.php` añade una acción de fila nativa en la lista legacy de carritos (PS 1.7/8). Instalación manual (método `override_v8` + copy), no `installOverrides()`.

3. API del core utilizada (verificada en tags 1.7.8.11 / 8.1.7 / 9.0.x)

Operación	Llamada
Añadir / subir cantidad	<code>Cart::updateQty(\$delta, \$id_product, \$id_pa, \$id_cust, 'up')</code>
Bajar cantidad	<code>Cart::updateQty(\$delta, ..., 'down')</code>
Quitar línea	<code>Cart::deleteProduct(\$id_product, \$id_pa, \$id_cust)</code>
Leer líneas	<code>Cart::getProducts(true)</code> (refresh obligatorio)
Totales	<code>Cart::getOrderTotal(true, Cart::BOTH ONLY_PRODUCTS ONLY_DISCOUNTS ONLY_SHIPPING)</code>
Unidades	<code>Cart::nbProducts()</code>
¿Es pedido?	<code>Order::getIdByCartId(\$id_cart)</code> (NO <code>getOrderByCartId</code> , eliminado en PS9)
Stock	<code>StockAvailable::getQuantityAvailableByProduct(\$id_product, \$id_pa)</code>
Validar stock	<code>Product::checkQty(\$qty)</code>
Precio	<code>Product::getPriceStatic(\$id_product, true, \$id_pa)</code>
Combinaciones	<code>Product::getAttributeCombinations(\$id_lang)</code>
Vale por código	<code>CartRule::getIdByCode(\$code) + checkValidity(\$ctx, false, true)</code>
Aplicar / quitar regla	<code>Cart::addCartRule(\$id) / Cart::removeCartRule(\$id)</code>
Crear descuento al vuelo	<code>new CartRule()</code> con <code>date_from</code> , <code>date_to</code> , <code>name</code> (obligatorios) + <code>reduction_percent</code> / <code>reduction_amount</code> + <code>reduction_currency</code> / <code>free_shipping</code>

Puntos críticos: - `updateQty` recibe el **delta**, no la cantidad absoluta, y puede devolver **-1** (cantidad por debajo del mínimo) → se comprueba `=== -1` y `!$result`. - Antes de `updateQty` / `checkValidity` se prepara el contexto: `customer`, `shop`, `currency` y `cart` (`prepareCartContext()`). - Los descuentos rápidos creados por el módulo (sin código, atados al cliente) se eliminan con `CartRule->delete()` al quitarlos, salvo que ya estén en un pedido (`order_cart_rule`).

4. Endpoints AJAX

`GetCart`, `SearchProducts`, `AddProduct`, `UpdateQuantity`, `RemoveProduct`, `ApplyVoucher`, `AddQuickDiscount` (tipos `percent` | `amount` | `shipping`), `RemoveCartRule`, `ToggleGift`. Todos devuelven `{ success, state }` o `{ success:false, error }`. Respuesta JSON vía `header()` + `echo json_encode()` + `exit` (compatible con PS9, donde `ajaxDie()` se eliminó).

5. Seguridad

- Token de controlador legacy validado automáticamente; la URL la genera `getAdminLink('AdminEcomEditCart')`.
- SQL parametrizado con `pSQL()` y casts `(int)`.
- Salida escapada en JS (`esc()`) y en Smarty (`|escape:'html':'UTF-8'`).
- Bloqueo por defecto de carritos ya convertidos en pedido (`ECOM_EDITCART_ALLOW_ORDERED = 0`).
- Registro de auditoría con `PrestaShopLogger::addLog(..., 'Cart', id, true)` incluyendo empleado, acción y carrito.

6. Configuración (Configuration)

Clave	Por defecto	Función
<code>ECOM_EDITCART_ENABLED</code>	1	Activa el módulo
<code>ECOM_EDITCART_INJECT_BTN</code>	1	Injecta el botón en la página de carritos
<code>ECOM_EDITCART_ALLOW_ORDERED</code>	0	Permite editar carritos ya convertidos en pedido
<code>GMARTOS_*</code>	—	Registro y aviso de actualización (ping semanal, cURL nativo)

7. Estructura de ficheros

<code>ecom_editcartadmin.php</code>	Clase principal (install/uninstall/hooks/config)
<code>controllers/admin/AdminEcomEditCartController.php</code>	Editor + endpoints AJAX
<code>views/templates/admin/editcart.tpl</code>	Shell del editor + aviso legal
<code>views/templates/admin/config_intro.tpl</code>	Intro + aviso legal en configuración
<code>views/templates/admin/gmartos_update.tpl</code>	Aviso de nueva versión
<code>views/js/editcart.js</code>	Lógica/render del editor (eventos delegados)
<code>views/js/editcart-inject.js</code>	Botón en la página nativa de carritos
<code>views/css/editcart.css</code>	Estilos
<code>override_v8/controllers/admin/AdminCartsController.php</code>	Acción de fila nativa (opcional, manual)
<code>docs/</code>	Documentación (3 .md)
<code>index.php</code>	Centinela en cada directorio

Edit Cart Admin — Guía para desarrolladores

Flujo general

1. El editor vive en el controlador `AdminEcomEditCartController`. `initContent()` monta el shell (Smarty) y publica la configuración a JS con `Media::addJsDef('ecomEditCart', [...])` (URL AJAX, token, id_cart inicial y textos).
2. `views/js/editcart.js` corre con `jQuery(document).ready` y **delegación de eventos sobre `document`** (los controles se renderizan dinámicamente, por eso no se usan binds directos). Al cargar, si hay `id_cart`, llama a `GetCart` y pinta el estado.
3. Cada acción mutadora (añadir, cantidad, quitar, vale, descuento, envío gratis, regalo) llama a su endpoint AJAX, que **devuelve el estado completo del carrito** (`buildCartState`) y el JS vuelve a renderizar todo. No hay estado duplicado entre PHP y JS.

Añadir un nuevo endpoint AJAX

```
public function ajaxProcessMiAccion()
{
    $cart = $this->loadEditableCart((int) Tools::getValue('id_cart')); // valida y corta si no
    editable
    // ... lógica con la API de Cart ...
    $this->prepareCartContext($cart);
    // ... mutación ...
    $this->logAudit('Mi accion', $cart, 'detalle=...');
    $this->respondJson(array('success' => true, 'state' => $this->buildCartState($cart)));
}
```

En JS, dispara con `ajax('MiAccion', { id_cart: getId(), ... }, function (res) { render(res.state); });`.

Reglas y convenciones (proyecto)

- Compatibilidad **PHP 7.1+**: sin typed properties en la clase principal, sin `declare(strict_types)`, sin sintaxis exclusiva de PHP 8 (enums, named args, promotion, union types, `match`, `?->`, `$obj::class`).
- Cabecera con autor `Ecom Experts <ecomyse@com>`, copyright 2026, AFL-3.0 en **todos** los ficheros; cero líneas en blanco antes de `<?php`.
- `index.php` centinela en cada directorio.
- `if (!defined('_PS_VERSION_')) exit;` tras la cabecera en todos los PHP.
- SQL con `pSQL()` / `(int)`; nada de `LIMIT 1` manual en `getValue` / `getRow`.
- Nuevas funcionalidades sin ficheros `upgrade`: usar `addNewFeatures()` (con try/catch) en `getContent()`.
- Sistema de traducción nuevo activado (`isUsingNewTranslationSystem()`), dominio `Modules.Ecomeditcartadmin.Admin`.

Integración con la página nativa de carritos

- **Primario (todas las versiones):** `hookDisplayBackOfficeHeader` inyecta `editcart-inject.js`, que detecta `sell/orders/carts` (Symfony 8/9) o `controller=AdminCarts` (legacy 1.7) y añade el botón leyendo el `id_cart` del enlace de "ver" o de la URL. Re-inyecta tras refrescos AJAX del grid con `MutationObserver`. Si cambia el markup en una versión, ajustar los selectores en `editcart-inject.js`.
- **Opcional nativo (PS 1.7/8):** copiar `override_v8/controllers/admin/AdminCartsController.php` a `override/controllers/admin/` y borrar `var/cache/*/class_index.php`. Añade `addRowAction('editcart')` + `displayEditcartLink`. En PS 9 la lista es Symfony puro y este override no aplica (funciona la inyección JS).

Pruebas recomendadas

1. Abrir el editor desde el menú **Pedidos** → **Editar carritos** y cargar un `id_cart`.
2. Buscar un producto por nombre y por referencia; comprobar imagen, combinaciones y botón **Añadir**.
3. Cambiar cantidades, quitar líneas, aplicar vale, descuento % e importe fijo, envío gratis y regalo; verificar que los totales se recalculan.
4. Cargar un carrito ya convertido en pedido: debe mostrar el aviso de contrato y bloquear la edición (salvo que se active la opción).
5. Revisar **Parámetros avanzados** → **Logs** para ver el registro de auditoría de cada cambio.

Puntos a verificar en la instalación real (marcados en la investigación)

- Selectores DOM del botón inyectado pueden variar entre 8.x y 9.x.
- `Tools::displayPrice()` sigue válido; si en una 9.x futura se marca deprecated, el formateo cae a `Locale::formatPrice` (ya contemplado en `formatPrice()`).
- El endpoint `ajaxProcessSearchProducts` usa `product_shop / product_lang.id_shop` (presentes en 1.7/8/9 estándar).